

빅데이터 분석 시험 대비 총정리 (실전 예시 중심)

1. 빅데이터 분석이란?

1.1. 빅데이터의 핵심: "일반화"

빅데이터 분석은 단순히 큰 데이터를 다루는 것을 넘어, 데이터로부터 의미 있는 모델과 요약물을 이끌어내는 "일반화 (Generalization)" 과정입니다.

- 두 가지 관점:
 1. 기술적 관점: "너무 커서 컴퓨터 한 대에 안 올라가는 데이터를 어떻게 분석할까?" (e.g., 분산 처리 기술)
 2. 분석적 관점: "수많은 데이터 속에서 어떻게 의미 있는 패턴을 찾을까?" (e.g., 통계, 머신러닝 알고리즘)
- 이를 위해 데이터 프레임워크(e.g., Hadoop, Spark)와 알고리즘(e.g., 추천 시스템, 딥러닝)을 함께 사용합니다.

1.2. 빅데이터 분석의 종류

- 기술 분석 (Descriptive Analytics): "과거에 무슨 일이 있었는가?" 를 요약하고 설명합니다.
 - 예시: 구글의 페이지랭크(수많은 웹페이지를 '중요도'라는 하나의 숫자로 요약), 의료 영상 분석(수백만 픽셀을 '정상 조직', '종양 의심' 등 몇 개 조직으로 요약).
- 예측 분석 (Predictive Analytics): "미래에 무슨 일이 일어날 것인가?" 를 예측합니다.
 - 예시: 트위터 감성 분석으로 주식 시장 예측, 사용자의 구매 기록으로 '함께 구매할 만한 상품' 추천.

1.3. 빅데이터 분석의 기본 도구 (핵심 개념)

이 강의에서 계속 사용될 핵심 아이디어들입니다.

1.3.1. 본페로니 교정 (Bonferroni's Principle)

- 문제점: 여러 가설을 동시에 검정하면, 우연히 틀린 결론(False Positive)을 내릴 확률이 급격히 증가합니다.
- 해결책: 개별 검정의 판단 기준(유의수준)을 훨씬 더 엄격하게 만듭니다. (e.g., 유의수준 5%로 10개 검정 시, 개별 기준을 $0.05/10 = 0.005$ 로 조정)
- 핵심: 데이터에서 무언가를 "발견"했다고 주장하려면, 그게 우연이 아니라는 것을 보장할 더 엄격한 잣대가 필요하다.

🔗 예상 문제: 본페로니 교정

한 연구팀이 웹사이트 사용자 로그를 분석하여, '버튼 색상', '버튼 위치', '광고 문구', '폰트 크기'가 클릭률에 영향을 주는지 총 4개의 독립적인 가설을 검정하려고 합니다. 전체적으로 잘못된 결론을 내릴 확률(Family-wise error rate)을 5% 미만으로 유지하고 싶을 때, 각 개별 가설 검정에 적용해야 할 유의수준(p-value의 기준값)은 얼마입니까?

🔗 상세 풀이 >

- 전체 허용 오류율 (α_{total}): 0.05 (5%)
- 검정 횟수 (n): 4
- 본페로니 교정에 따라, 수정된 유의수준 $\alpha' = \alpha_{total} / n$ 입니다.
- $\alpha' = 0.05 / 4 = 0.0125$
- 결론: 각 개별 가설은 p-value가 0.0125보다 작을 때만 "통계적으로 유의미하다"고 판단해야 합니다.

1.3.2. 정규화 (Normalization) & TF-IDF

- 필요성: 여러 변수(feature)들의 값의 범위(scale)가 다르면, 특정 변수가 분석 결과를 왜곡할 수 있습니다. (e.g., 나이: 0~100, 연봉: 0~1억)
- 방법: 모든 변수를 비슷한 범위(e.g., 0~1)나 분포(e.g., 평균 0, 표준편차 1)로 맞춰주는 작업.

- 대표적인 예: **TF-IDF**

- **TF (Term Frequency):** 한 문서 안에서 특정 단어가 얼마나 자주 나오는가? (문서 내 중요도)
- **IDF (Inverse Document Frequency):** 그 단어가 전체 문서들 중 얼마나 희귀한가? (전체에서 본 중요도)
- **TF-IDF:** TF와 IDF를 곱한 값. 특정 문서에서만 자주 등장하는, 즉 그 문서를 잘 대표하는 단어일수록 높은 값을 가집니다.

🔗 예상 문제: **TF-IDF 계산**

다음과 같은 3개의 문서가 있다고 가정합니다.

- doc1 : "스파크는 빠르다 스파크는 대세다"
- doc2 : "하둡은 스파크보다 느리다"
- doc3 : "하둡 맵리듀스는 대세였다"

문제: doc1 에서 단어 "스파크"의 TF-IDF 점수를 계산하십시오. (log는 자연로그 $\ln$ 을 사용)

🔗 상세 풀이 >

1. TF (Term Frequency, 단어 빈도) 계산

TF는 (특정 단어가 문서에 나타난 횟수) / (문서의 전체 단어 수) 입니다.

- doc1 의 전체 단어 수: 4개 ("스파크", "빠르다", "스파크", "대세")
- doc1 에서 "스파크"의 등장 횟수: 2번
- $TF("스파크", doc1) = 2 / 4 = 0.5$

2. IDF (Inverse Document Frequency, 역문서 빈도) 계산

IDF는 $\ln((총 문서의 수) / (해당 단어가 등장한 문서의 수))$ 입니다.

- 총 문서의 수: 3개 (doc1 , doc2 , doc3)
- 단어 "스파크"가 등장한 문서 수: 2개 (doc1 , doc2)
- $IDF("스파크") = \ln(3 / 2) = \ln(1.5) \approx 0.405$

3. TF-IDF 점수 계산

$TF-IDF = TF * IDF$

- $TF-IDF("스파크", doc1) = 0.5 * 0.405 = 0.2025$

결론: doc1 에서 단어 "스파크"의 TF-IDF 점수는 약 0.2025 입니다.

1.3.3. 멱법칙 (Power Laws)

- 정의: "큰 사건은 드물고, 작은 사건은 자주 발생한다"는 패턴. (a.k.a. 롱테일, 부익부 빈익빈)
- 예시: 소수의 베스트셀러가 서점 전체 매출의 대부분을 차지, 상위 1% 유튜버가 전체 조회수의 대부분을 차지.
- 중요성: 현실 세계의 많은 데이터는 정규분포가 아닌 멱법칙을 따르므로, 극단값의 존재를 인지하고 모델링해야 합니다.

1.3.4. 해시 함수 (Hash Functions) & 스트리밍 알고리즘

- 정의: 임의의 길이의 데이터를 고정된 길이의 데이터(해시값)로 바꿔주는 함수.
- 빅데이터에서의 역할: 데이터를 특정 그룹(버킷)에 효율적으로 나누어 담거나, 데이터베이스 인덱싱을 통해 검색 속도를 $O(1)$ 수준으로 향상시키는 데 필수적.

🔗 예상 문제: **스트리밍 알고리즘**

Part (a) - 블룸 필터:

크기 8의 비트 배열과 2개의 해시 함수 $h_1(x) = x \% 8$, $h_2(x) = (x \% 5) + 1$ 가 있습니다. 집합에 [10, 19] 두 개의 원소를 추가했을 때, test_item 으로 4 가 들어오면 **False Positive**가 발생하는지 확인하십시오.

③ 상세 풀이 (a) >

1. 초기 비트 배열: $[0, 0, 0, 0, 0, 0, 0, 0]$
2. 원소 추가:
 - 10 추가: $h_1(10)=2$, $h_2(10)=1$ -> 1번, 2번 인덱스를 1로 변경. $[0, 1, 1, 0, 0, 0, 0, 0]$
 - 19 추가: $h_1(19)=3$, $h_2(19)=5$ -> 3번, 5번 인덱스를 1로 변경. $[0, 1, 1, 1, 0, 1, 0, 0]$
3. 4 검사:
 - $h_1(4) = 4 \% 8 = 4$
 - $h_2(4) = (4 \% 5) + 1 = 5$
 - 비트 배열의 4번과 5번 인덱스를 확인합니다.
 - 배열[4]는 0이고, 배열[5]는 1입니다.
4. 결론: 검사할 위치 중 하나(4번 인덱스)가 0이므로, 블룸 필터는 '없다'고 올바르게 판단합니다. 따라서 **False Positive**가 발생하지 않습니다.

Part (b) - Flajolet-Martin:

어떤 데이터 스트림의 원소들을 해싱한 결과, 각 해시값의 끝에 붙는 0의 개수($R(x)$)가 $\{1, 0, 3, 2, 0, 4, 1, 2\}$ 로 나왔습니다. 이 스트림의 고유 원소 개수(Cardinality)를 추정하십시오.

③ 상세 풀이 (b) >

1. **R_max** 찾기: $R(x)$ 값들 중 최댓값을 찾습니다.
 - $R_{\max} = \max(\{1, 0, 3, 2, 0, 4, 1, 2\}) = 4$
2. 고유 원소 수 추정: 추정값은 $2^{R_{\max}}$ 입니다.
 - 추정값 $= 2^4 = 16$

결론: 추정된 고유 원소의 개수는 **16개**입니다.

1.3.5. One-Hot Encoding

- 정의: 'red', 'green', 'blue'와 같은 범주형 데이터를 컴퓨터가 계산할 수 있는 수치 데이터(벡터)로 변환하는 기법.
- 방법: 각 범주마다 고유한 차원을 할당하고, 해당하는 범주만 1, 나머지는 0으로 채웁니다. (e.g., Red: $[1, 0, 0]$, Green: $[0, 1, 0]$)
- 장점: 모델이 범주 간에 존재하지 않는 순서(e.g. Red < Green)를 잘못 학습하는 것을 방지합니다.

2. 하둡(Hadoop): 분산 아키텍처와 MapReduce

2.1. 왜 '분산'을 해야 하는가?

빅데이터의 가장 큰 현실적인 문제는 데이터가 너무 커서 한 대의 컴퓨터에 다 안 들어간다는 점과 디스크에서 데이터를 읽는 속도가 너무 느리다는 점입니다.

- 실전 예시 (I/O 병목 현상): 200TB(테라바이트)짜리 데이터를 분석해야 한다면, 데이터를 디스크에서 메모리로 읽어오는 데만 수십 일이 걸릴 수 있습니다. CPU는 놀고 있는데 데이터 로딩만 기다리는 상황이 바로 I/O 병목입니다.
- 해결책 (분산 아키텍처): "혼자서 100일 걸릴 일을 100명이 나눠서 하면 하루 만에 끝낼 수 있다!" -> 저렴한 컴퓨터 여러 대를 묶어(클러스터) 일을 나누어 처리합니다.

2.2. 분산 처리의 현실 문제와 '하둡'의 해결책

1. 문제: "컴퓨터가 고장 나면 데이터가 날아간다!"

- 하둡의 해결책 (**HDFS** - 데이터 복제): 데이터를 저장할 때부터 원본 데이터 조각을 **2개** 더 복제해서 총 **3개**를 서로 다른 컴퓨터에 나눠서 저장합니다. 하나가 고장나도 다른 복제본을 사용해 데이터를 복구합니다 (내고장성, Fault-Tolerance).
2. 문제: "데이터를 분석하려고 한 곳으로 옮기는 시간이 더 걸린다!"
- 하둡의 해결책 (**MapReduce** - 계산 로직 이동): "데이터야, 너는 가만히 있어. 내가 계산할 코드를 네가 있는 곳으로 보내줄게." -> 각 컴퓨터가 자신이 가진 데이터만 분석하고 결과만 중앙으로 보냅니다.
3. 문제: "분산 프로그래밍은 너무 복잡하고 어렵다!"
- 하둡의 해결책 (**MapReduce** - 프로그래밍 모델 단순화): 개발자는 **Map** 함수와 **Reduce** 함수만 만들면, 하둡이 데이터 분배, 코드 배포, 장애 처리, 결과 취합 등 모든 복잡한 과정을 자동으로 처리해줍니다.

2.3. MapReduce 작동 방식: '단어 개수 세기' 예시

"분할 정복(Divide and Conquer)" 철학을 따르는 "맵(Map) -> 셔플(Shuffle) -> 리듀스(Reduce)" 3단계 프로세스입니다.

1. **Map** 단계 (분류/전처리):
 - 역할: 원본 데이터를 의미 있는 (Key, Value) 쌍으로 만드는 단계.
 - 예시: 텍스트 한 줄을 받아, 그 안의 모든 단어에 대해 (단어, 1) 이라는 (Key, Value) 쌍을 만들어냅니다. ("To be", 1) -> (to, 1), (be, 1)
2. **Shuffle & Sort** 단계 (그룹화):
 - 역할: 모든 map 결과물들을 Key 를 기준으로 재분배하고 정렬하여 그룹화하는 단계. (시스템 자동 처리)
 - 예시: 전 세계에 흩어져 있던 (to, 1) 들을 한 곳으로 모아 (to, [1, 1, 1, ...]) 형태로 만듭니다.
3. **Reduce** 단계 (최종 집계):
 - 역할: 그룹화된 데이터를 가지고 실제 계산을 수행하는 단계.
 - 예시: (to, [1, 1, 1, 1]) 를 입력받아, 리스트 안의 모든 1을 더해서 최종 결과 (to, 4) 를 출력합니다.

3. 스파크(Spark): 빠르고 유연한 차세대 빅데이터 엔진

3.1. 왜 스파크인가? - 하둡 MapReduce의 한계 극복

- 하둡의 치명적 단점: 모든 작업 단계마다 중간 결과를 디스크에 쓰고 읽습니다. 이는 반복 작업(머신러닝)이나 실시간 분석에서 매우 느립니다.
- 스파크의 해결책: "중간 결과를 디스크가 아니라 메모리에 저장하자!" -> 훨씬 빠르고(최대 100배), 더 다양한 분석 작업을 하나의 엔진으로 처리할 수 있습니다.

3.2. 스파크의 핵심 개념: RDD (Resilient Distributed Dataset)

스파크의 모든 것은 **RDD**라는 개념 위에 세워져 있습니다.

- 정의: "장애 복구가 가능한 분산 데이터 셋"
- 핵심 특징 (**Resilient**): RDD는 데이터 자체뿐만 아니라, "이 데이터가 어떤 과정을 거쳐 만들어졌는지"에 대한 정보(계보, **Lineage**)를 모두 기억합니다. 데이터가 유실되어도, 이 계보를 통해 원본 데이터로부터 자동으로 복구할 수 있습니다.
- **RDD**의 두 가지 연산 (매우 중요):
 1. **Transformation (변환)**: 새로운 RDD를 만드는 연산. `map`, `filter` 등이 있으며, 게으르게 실행(**Lazy Evaluation**)됩니다. (호출 즉시 실행되지 않고 계획만 세웁니다.)
 2. **Action (액션)**: 실제 계산을 촉발하는 연산. `count()`, `collect()` 등이 있으며, 이 연산이 호출될 때 그동안 계획했던 모든 변환 작업이 한꺼번에 실행됩니다.

3.3. 스파크 프로그래밍 핵심 요소

- **cache() (persist())**: 스파크 성능의 핵심. RDD를 메모리에 저장하여 이후의 Action에서 재사용하게 함으로써 반복 작업의 속도를 획기적으로 높입니다.
 - 실전 예시: 머신러닝 학습 데이터를 `cache()` 해두면, 여러 파라미터로 모델을 반복 학습시킬 때 매번 디스크에서 데이터를 읽어오는 비효율을 막을 수 있습니다.

- **Broadcast Variables** (읽기 전용 공유 변수): 모든 작업 노드에 딱 한 번만 큰 데이터를 효율적으로 전송하고 싶을 때 사용합니다. (e.g., 수백만 개의 단어가 담긴 '스팸 단어 사전'을 배포할 때)
- **Accumulators** (누적기): 분산된 환경에서 모든 노드들이 각자 계산한 값을 안전하게 합산하고 싶을 때 사용합니다. (e.g., 에러 라인 총 개수, 빈 줄 개수 등을 셀 때)

4. 데이터 통계 기초

데이터 분석은 통계라는 무기 없이는 불가능합니다.

4.1. 확률 변수와 확률 분포

- **확률 변수 (Random Variable, X)**: 사건(e.g., '카드 뽑기')을 컴퓨터가 계산할 수 있는 숫자로 바꿔주는 함수.
- **PMF** (확률 질량 함수): 셀 수 있는 값(이산 데이터)이 특정 숫자가 될 확률. (이산적인 히스토그램)
- **CDF** (누적 분포 함수): 확률 변수가 특정 값보다 작거나 같을 확률. (계단 모양 그래프)
- **기댓값 (Expectation)**: 확률을 고려한 평균. (값 * 확률) 의 총합.

4.2. 실전에서 자주 만나는 확률 분포 모델들

1. **베르누이 분포**: 결과가 '성공/실패' 등 딱 두 가지인 한 번의 시행. (e.g., 광고 클릭/미클릭)
2. **이항 분포**: 베르누이 시행을 n 번 반복했을 때, 성공이 총 k 번 나올 확률. (e.g., 10개 메일 중 8개를 스팸으로 예측할 확률)
3. **기하 분포**: 첫 성공까지 시도한 횟수에 대한 확률. (e.g., 희귀 아이템을 뽑을 때까지 시도하는 횟수)
4. **포아송 분포**: 정해진 시간/공간 동안 평균 λ 번 일어나는 사건이, 이번에는 k 번 일어날 확률. (e.g., 1시간 동안 콜센터에 전화가 5통 올 확률, 우버 배차 시스템)
5. **정규 분포**: 평균을 중심으로 좌우대칭인 종 모양 분포. 세상의 가장 흔한 데이터 패턴. (e.g., 사람들의 키 분포)

🔗 예상 문제: 카이제곱 검증

어떤 쇼핑몰에서 A, B, C 세 가지 다른 디자인의 배너 광고를 걸었을 때, 기대 클릭 수는 모두 동일하게 30회였습니다. 하지만 실제 관측된 클릭 수는 각각 A: 40회, B: 30회, C: 20회였습니다. 이 차이가 통계적으로 유의미한지 유의수준 5%에서 검정하시오. (단, 자유도 2일 때 카이제곱 임계값은 5.99라고 주어짐)

🔗 상세 풀이 >

1. 가설 설정:

- **귀무가설 (H_0)**: 배너 디자인과 클릭 수 사이에는 관계가 없다 (관측값과 기대값은 우연히 차이가 난 것뿐이다).
- **대립가설 (H_1)**: 배너 디자인과 클릭 수 사이에는 관계가 있다 (차이가 유의미하다).

2. 카이제곱 통계량(χ^2) 계산:

$$\chi^2 = \sum [(\text{관측값} - \text{기대값})^2 / \text{기대값}]$$

- 배너 A: $(40 - 30)^2 / 30 = 100 / 30 \approx 3.33$
- 배너 B: $(30 - 30)^2 / 30 = 0 / 30 = 0$
- 배너 C: $(20 - 30)^2 / 30 = 100 / 30 \approx 3.33$
- χ^2 (총합) = $3.33 + 0 + 3.33 = 6.66$

3. 결론:

- 계산된 카이제곱 통계량(6.66)은 주어진 임계값(5.99)보다 큼니다.
- 통계량이 임계값보다 크므로 귀무가설을 기각합니다. 즉, 배너 디자인에 따른 클릭 수의 차이는 단순한 우연이 아니며 통계적으로 유의미하다고 결론 내릴 수 있습니다.

5. 변수 선택 (Feature Selection)

"쓰레기를 넣으면 쓰레기가 나온다 (Garbage In, Garbage Out)". 좋은 예측 모델을 만들려면 좋은 재료(변수)를 골라야 합니다.

5.1. 왜 변수 선택을 하는가?

- 다중공선성 (**Multicollinearity**) 문제 해결: 서로 너무 비슷한 정보를 가진 변수들(e.g., '방 개수'와 '집 평수')이 함께 있으면 모델 성능이 저하됩니다.
- 효과: 과적합 방지, 계산 속도 향상, 모델 해석 용이.

5.2. 변수 선택 기법들 (PySpark 기준)

1. 타겟 변수 없이 선택 (Unsupervised)

- 주성분 분석 (**PCA**): 여러 변수에 흩어진 정보를 가장 잘 대표하는 새로운 '대표 변수'(주성분)를 만들어 차원을 축소.
- 특이값 분해 (**SVD**): 데이터에 숨겨진 잠재적인 의미(**Latent Factor**)를 찾아 차원을 축소. (e.g., 영화 추천 시스템에서 'SF 컨셉', '로맨스 컨셉'을 찾아내는 것)

🔗 예상 문제: SVD와 차원 축소

어떤 (사용자 x 영화) 평점 행렬을 SVD로 분해했더니, 특이값(singular value)이 각각 $\sigma_1=15$, $\sigma_2=10$, $\sigma_3=3$, $\sigma_4=1$, $\sigma_5=0.5$ 로 나왔습니다. 데이터의 에너지(분산)를 **95% 이상** 보존하려면 최소 몇 개의 특이값(차원)을 선택해야 하는지 계산하십시오.

🔗 상세 풀이 >

1. 전체 에너지 계산: 에너지는 특이값의 제곱에 비례합니다.
 - 전체 에너지 = $15^2 + 10^2 + 3^2 + 1^2 + 0.5^2$
 - = $225 + 100 + 9 + 1 + 0.25 = 335.25$
2. 특이값 순서대로 에너지 누적 비율 계산:
 - **k=1** (σ_1 만 사용): $225 / 335.25 \approx 67.1\%$ (95% 미만)
 - **k=2** (σ_1, σ_2 사용): $(225 + 100) / 335.25 = 325 / 335.25 \approx 96.9\%$ (95% 이상)
3. 결론:
 - 상위 2개의 특이값만 사용해도 데이터 에너지의 약 96.9%를 보존할 수 있으며, 이는 목표인 95%를 넘습니다.
 - 따라서 최소 **2개**의 특이값을 선택해야 합니다.

2. 타겟 변수를 활용하여 선택 (Supervised)

- 카이제곱 선택 (**ChiSq Selector**): 카이제곱 검정으로 각 변수가 타겟 변수와 관련이 있는지 통계적으로 평가하여 관련 없는 변수를 제거.
- 모델 기반 선택 (**Model-based Selection**): 랜덤 포레스트 같은 트리 기반 모델을 학습시켜, 예측에 기여한 정도(변수 중요도, `feature_importances_`)를 기준으로 상위 변수들을 선택.
- 정보 값 (**Information Value, IV**) & 증거 가중치 (**Weight of Evidence, WoE**):
 - 신용 점수 모델링에서 유래한 기법으로, 각 변수의 구간별로 불량/정상 비율을 계산하여 변수의 예측력을 수치화합니다.
 - **WoE**: 특정 구간이 전체 평균 대비 얼마나 불량(또는 정상) 고객을 잘 잡아내는지 나타내는 값. $\ln(\text{구간 내 정상 비율} / \text{구간 내 불량 비율})$
 - **IV**: 각 구간별 WoE 값을 가중합하여 변수 전체의 예측력을 하나의 숫자로 요약한 것. 일반적으로 IV가 0.1 ~ 0.3 이면 '중간', 0.3 이상이면 '강한' 예측력을 가진 변수로 판단합니다.

5.3. 파이프라인 (Pipeline)

문자열 변환(`StringIndexer`), 원-핫 인코딩, 특징 벡터화(`VectorAssembler`), 스케일링, 모델 학습 등 전체 데이터 처리와 분석 과정을 하나의 작업 흐름(**Workflow**)으로 묶어주는 스파크의 강력한 기능입니다. 이를 통해 전체 분석 과정을 체계적으로 관리하고 재사용할 수 있습니다.